

recall a certain attribute of an entity correctly (i.e., assign a high logit value to the respective token), we diagnose the behavior as *factually self-aware*.

We feed the samples (entity-relation pairs) into an LM to obtain the probability distribution over the predicted tokens, and classify the factual relations as either *known* or *forgotten*. Since the gold labels are sequences of tokens (e.g., “Christopher”, “Nolan”) we generate the same number of tokens as in the gold label sequence for each sample and check how many of these appear in the top- k or bottom l -th percentile of the model’s output space³. A sample is labeled as *known* if more of its gold label tokens appear among the top- k predictions in the logit space. Conversely, if more of the gold label tokens fall below the l -th percentile of the logit distribution, the sample is classified as *forgotten*. This design choice avoids decoding and string matching errors by relying solely on model’s output space.

We construct (entity type, entity name, relation) triplets using various templates, but some templates introduced spurious correlations due to their phrasing. Details and examples of all templates are provided in Table 3 and Appendix A. For subsequent experiments, we employ a template devoid of such artifacts, encompassing four relations per entity type and excluding problematic attributes (e.g., city coordinates). The complete list of relations is provided in Appendix A.

We construct the final dataset by setting top- $k = 500$ tokens and selecting the bottom- $l = 0.3$ fraction of the vocabulary space. Detailed experiments on the impact of different (k, l) pairs on factual self-awareness signals are presented in subsection 4.3. This procedure yields a total of 7,380 *known* and 7,268 *forgotten* samples for Gemma 2 2B Team et al. (2024). The distribution of labels across entity types is provided in Table 4 in Appendix B. We partition the dataset $\mathcal{D}$ into training and test subsets with a 0,7/0,3 split for subsequent experiments.

Linear Probe. Let $\mathcal{D} = \{(T_i, y_i)\}_{i=1}^N$ denote the labeled dataset, where each T_i is a token sequence (e.g., a factual recall statement) and $y_i \in \{0, 1\}$ indicates the presence or absence of the feature (e.g., *known/forgotten*). We run the model on each $T_i \in \mathcal{D}$ and extract the residual stream of the final token of the prompt template x_{T_i} , following Meng et al. (2022); Geva et al. (2023); Nanda et al. (2023).

Definition 1. For a residual stream representation $x_{l,T_i} \in \mathbb{R}^d$ of sample T_i at layer l , a probe is a learnable function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ trained to predict y_i from x_{l,T_i} .

The linear probe serves as a simple diagnostic classifier that maps the residual stream output to a scalar output via a learned weight vector $\mathbf{w} \in \mathbb{R}^d$ and a bias term $b \in \mathbb{R}$. At layer l , we introduce a linear probe and its corresponding optimization objective as:

$$f_l : \mathbb{R}^d \rightarrow \mathbb{R}, \quad f_l(x_{l,T_i}) = \mathbf{w}^\top x_{l,T_i} + b, \quad \min_{\mathbf{w}, b} \sum_{(T_i, y_i) \in \mathcal{D}} \text{BCE}(y_i, \sigma(f_l(x_{l,T_i}))). \quad (1)$$

where σ is the sigmoid function and BCE the binary cross-entropy loss. The parameters $\mathbf{w}$ and b are learned by minimizing the binary cross-entropy loss over the dataset $\mathcal{D}$.

To break symmetry and introduce controlled variation across layers, we initialize scalar biases as $b = 0.1 \times (-1)^l$, where the layer index l deterministically seeds randomness via `seed + 100 × l` with `seed`⁴. This ensures consistent yet diverse initialization across layers. To further encourage diversity in learned solutions, each probe is assigned a slightly different learning rate, scaled for layer l as $\text{lr}_l = \text{base_lr} \times (1.1 - 0.2 \cdot \frac{l}{L})$, where `base_lr` is the initial rate and L the total number of layers. This encourages probes at different depths to converge to distinct solutions. All probes are optimized using Adam, with initial learning rate $1\text{e-}4$ and weight decay $1\text{e-}5$.

Separation Scores. Sparse Autoencoders (SAEs) conform to the definition of a probe as stated in Definition 1. We collectively refer to the SAE-encoded representations and the outputs of the linear probe as *activations*. Following Ferrando et al. (2024), we compute separation scores. For each latent dimension j of the activation vector (with $j = 1$ for a linear probe), we calculate the proportion of

³Note that in this definition, k is an integer denoting the count of tokens, while l is a fraction denoting a subset of the vocabulary. This demarcation arises from the actual count of tokens in these bands: high probability tokens are exponentially fewer than low probability ones.

⁴All experiments are conducted with three random seeds (73, 5, 120); we observe negligible variance and omit the results for brevity.

instances with positive activations (i.e., greater than zero) separately for the known and forgotten sets: $g_{l,j}^{\text{known}} = \frac{\sum_i^{N^{\text{known}}} \mathbb{1}[a_{l,j}(x_{l,T_i}^{\text{known}}) > 0]}{N^{\text{known}}}$ and $g_{l,j}^{\text{forgotten}} = \frac{\sum_i^{N^{\text{forgotten}}} \mathbb{1}[a_{l,j}(x_{l,T_i}^{\text{forgotten}}) > 0]}{N^{\text{forgotten}}}$, where N^{known} and $N^{\text{forgotten}}$ denote the total number of prompts, and x_{l,T_i}^{known} and $x_{l,T_i}^{\text{forgotten}}$ represent the latent activations for the known and forgotten samples, respectively, in each subset of $\mathcal{D}$. Latent separation scores (or vectors) are computed as the difference between these proportions: $s_{l,j}^{\text{known}} = g_{l,j}^{\text{known}} - g_{l,j}^{\text{forgotten}}$ and $s_{l,j}^{\text{forgotten}} = g_{l,j}^{\text{forgotten}} - g_{l,j}^{\text{known}}$, where s_l^{known} is used to detect known entities, and $s_l^{\text{forgotten}}$ is used to detect forgotten entities.

Computational Resource Requirements. All experiments are conducted on NVIDIA A100-SXM4-80GB GPUs. Models with less than 7B parameters are run on a single GPU, while larger models are executed on two GPUs to accommodate memory and computational requirements.

4 Generation-time factual self-awareness in LMs

4.1 Linear Probes vs. Sparse Autoencoders (SAEs)

We utilize the Gemma 2 2B and 9B models from the Gemma Scope framework (Lieberum et al., 2024), which provides a suite of SAEs pretrained on the activations of each layer of the Gemma 2 models (Team et al., 2024). We train linear probes on the residual stream hooks of these models and generate the corresponding separation plots on test sets for both the 2B and 9B variants using both SAE and linear probing methods. We select the top-five (one for linear probe) latent dimensions with the highest separation scores from the known and forgotten vectors for each entity type t and layer l . To assess generality and robustness, we compute $\text{MaxMin}^{\text{known},l} = \max_j \min_t s_{l,j}^{\text{known},t}$ and analogously define $\text{MaxMin}^{\text{forgotten},l}$, where j indexes latent dimensions.

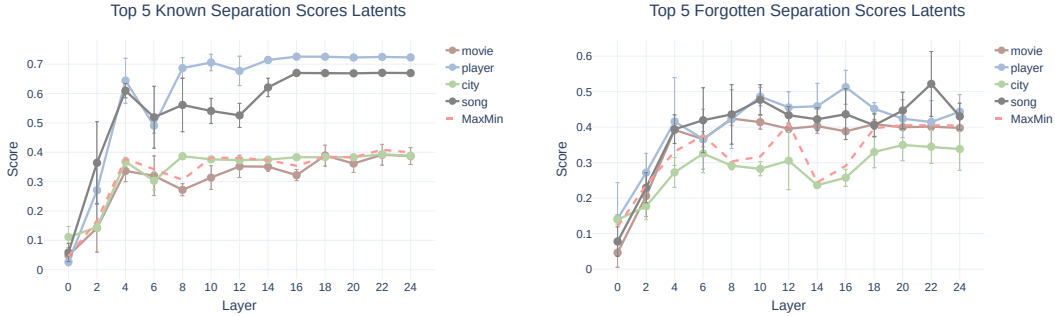


Figure 2: Top-five latent separation scores across transformer layers using SAE activations from Gemma 2 2B. Left: For **known** entities exhibit clear layer-wise separation, peaking around layers 6–14. Right: For **forgotten** entities, separation scores are lower and more variable, indicating reduced disentanglement. Categories include movie, player, city, and song; MaxMin denotes the difference between max and min class means.

We illustrate the evolution of separation scores across layers for SAE activations in Figure 2 (Gemma 2 9B model results are provided in Appendix C). As shown by the red curve, MaxMin_l increases in the intermediate layers. This pattern suggests that the most *generalized* latents—those consistently separating known from forgotten entities across all types are primarily located in the middle and final layers. Linear probe separation scores follow a similar trend, as shown in Figure 3; however, since these activations are scalar, the known and forgotten scores appear as mirror images, having equal magnitudes but opposite signs.

We train linear probes on the Gemma 2 (2B, 9B) (Team et al., 2024) and Pythia (70M, 1.4B, 6.9B, 12B) (Biderman et al., 2023) models, evaluating performance using standard binary classification metrics and reporting the accuracy improvement over a random baseline, denoted as Δ . Metrics from the final training epoch are reported for both the training and test subsets, as shown in Table 1. Among all models, Gemma 2 2B exhibits the strongest performance, achieving the highest test set separation score with $\Delta = 0.311$. Within the Pythia family, the 12B model performs best ($\Delta = 0.120$); however, a substantial performance gap remains relative to Gemma 2 2B.

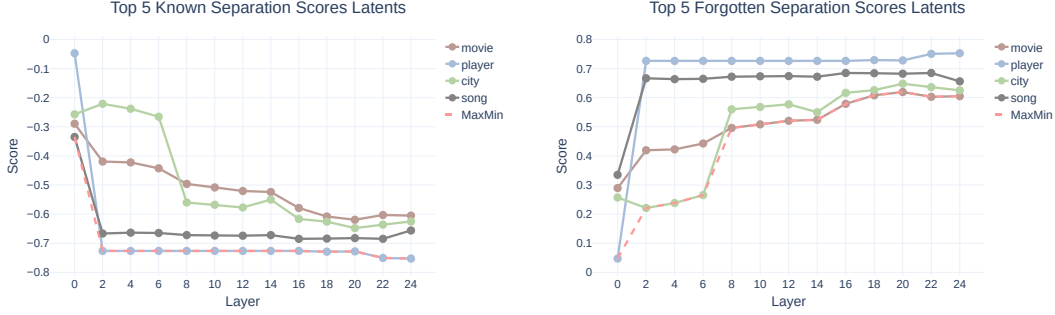


Figure 3: Latent separation scores across layers using Linear Probe activations from Gemma 2 2B. Left: **Known** entities show separation scores that are identical in magnitude but negated in sign compared to **forgotten** entities (right), indicating that the same latents are used for both but with reversed class-directional structure. Categories include movie, player, city, and song; MaxMin denotes the difference between maximum and minimum class means.

Table 1: Linear probing results on Gemma 2 and Pythia model families. Metrics are reported on training and test subsets from the final epoch (3). Accuracy gains over random baselines are indicated by Δ , with values in parentheses denoting standard deviations. Gemma 2 2B achieves the highest test set performance, while Pythia 12B is strongest within its family, though with a notable gap.

Model	Subset	Loss	AUC ROC	Accuracies		
				Observed	Random Baseline	Δ (Observed - Baseline)
Gemma 2 2B	Train	0.383 (0.063)	0.901 (0.061)	0.833 (0.052)	0.501	0.332
	Test	0.397 (0.064)	0.896 (0.060)	0.820 (0.056)	0.509	0.311
Gemma 2 9B	Train	0.393 (0.052)	0.899 (0.050)	0.826 (0.040)	0.555	0.271
	Test	0.387 (0.050)	0.903 (0.047)	0.829 (0.032)	0.564	0.265
Pythia 70M	Train	0.473 (0.008)	0.546 (0.029)	0.818 (0.001)	0.818	0.000
	Test	0.465 (0.005)	0.551 (0.022)	0.822 (0.001)	0.822	0.000
Pythia 1.4B	Train	0.358 (0.032)	0.843 (0.057)	0.837 (0.011)	0.807	0.030
	Test	0.365 (0.031)	0.842 (0.056)	0.831 (0.013)	0.803	0.028
Pythia 6.9B	Train	0.393 (0.028)	0.852 (0.048)	0.829 (0.014)	0.747	0.082
	Test	0.395 (0.026)	0.857 (0.047)	0.827 (0.011)	0.746	0.081
Pythia 12B	Train	0.442 (0.030)	0.842 (0.046)	0.798 (0.017)	0.687	0.111
	Test	0.464 (0.027)	0.846 (0.043)	0.794 (0.022)	0.674	0.120

Beyond the overall advantage of the Gemma 2 2B model, several notable trends emerge from the probing results. Within each model family, increasing parameter count does not consistently improve linear probe performance. For instance, while Gemma 2 9B achieves a slightly higher test AUC-ROC than Gemma 2 2B, its accuracy gain over the random baseline (Δ) is lower. This suggests that larger models do not necessarily produce more linearly decodable representations of self-awareness. A similar pattern holds for the Pythia models: although the 12B variant achieves the highest test-time Δ , smaller versions like Pythia 6.9B and 1.4B show comparable accuracies, albeit with smaller gains over their baselines. Pythia 70M represents a degenerate case where accuracy matches the random baseline ($\Delta = 0$), indicating that the smallest model fails to encode self-awareness features.

We further examine the distribution of self-awareness signals across model layers by analyzing layer-wise linear probe accuracy, as shown in Figure 4. For both Gemma 2 2B and Pythia 12B, accuracy rises sharply in the initial layers before stabilizing. In Gemma 2 2B, performance plateaus around the fifth layer, reaching a peak test accuracy of approximately 0.82; well above the random baseline. Accuracy remains consistently high in subsequent layers, with a slight decline in the final three layers, suggesting that self-awareness directions are preserved throughout the network depth.

In contrast, Pythia 12B shows a slower but steady accuracy increase across layers. Its final test accuracy remains below that of Gemma 2 2B, consistent with earlier results. Notably, the random baseline for Pythia 12B is substantially higher than for Gemma 2 2B (approximately 0.69 vs. 0.50). These patterns suggest that Gemma 2 2B achieves linearly accessible self-awareness representations earlier and maintains them more robustly, while Pythia 12B requires deeper processing to approach similar performance.